

Backend Fundamentals

A REST API exposes resources over HTTP using verbs such as GET, POST, PUT, and DELETE. Well-designed endpoints are stateless: each request carries the context needed to process it, which makes horizontal scaling straightforward because any server can handle any request.

Idempotency is a key property for reliable APIs. An idempotent operation can be repeated without changing the result beyond the first application, so clients can safely retry after a timeout. PUT and DELETE are naturally idempotent; POST usually is not unless an idempotency key is supplied.

Caching improves latency and reduces load by reusing previously computed results. Caches can live at many tiers: the client, a CDN, a reverse proxy, or an in-memory store such as Redis. Cache invalidation is famously hard; time-to-live expiry and explicit purges are the common strategies.

Scaling and Data

Load balancing spreads incoming traffic across a pool of servers so no single node is overwhelmed. Round-robin, least-connections, and consistent hashing are common algorithms. Health checks remove unhealthy instances from rotation automatically.

Sharding partitions a dataset across multiple database nodes so each holds only a slice of the data. It increases write throughput and storage capacity, but complicates queries that span shards and makes strong consistency harder. Choosing a good shard key is critical to avoid hot spots.

Consistency models describe how and when replicas agree. Strong consistency guarantees every read sees the latest write, while eventual consistency allows temporary divergence for higher availability. The CAP theorem frames the trade-off between consistency, availability, and partition tolerance.